

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250404>

Outline : Reinforcement learning {TF support} cont. (code [samples](#))

- Definitions / Motivation
- Markov Decision Processes
- Temporal Difference Learning
- Q-Learning
- Implementing Deep Q-Learning
- Deep Q-Learning Variants
- The TF-Agents Library
- Overview of Some Popular RL Algorithms
- Takeaways

—

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapter 18**

Definition / Motivation (1)

Playing Atari with Deep Reinforcement Learning (Videos)

Human-level control through deep reinforcement learning

In Reinforcement Learning, a software *agent* makes observations and takes *actions* within an *environment*, and in return it receives *rewards*. Its objective is to learn to act in a way that will maximize its expected rewards over time.

Definition / Motivation (2)

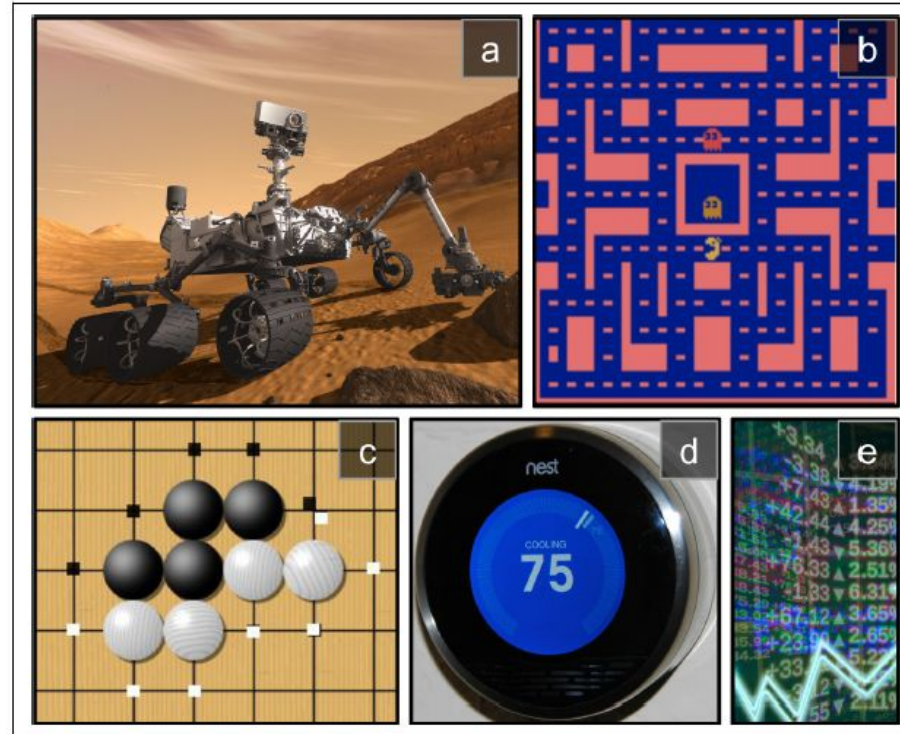


Figure 18-1. Reinforcement Learning examples: (a) robotics, (b) Ms. Pac-Man, (c) Go player, (d) thermostat, (e) automatic trader⁵

Markov Decision Processes

Markov Decision Processes (1)

- *Markov chains*: stochastic processes with no memory.
 - Fixed number of states, and it randomly evolves from one state to another at each step.
 - The probability for it to evolve from a state s to a state s' is fixed, and it depends only on the pair (s, s') , not on past states (so, the system has no memory).

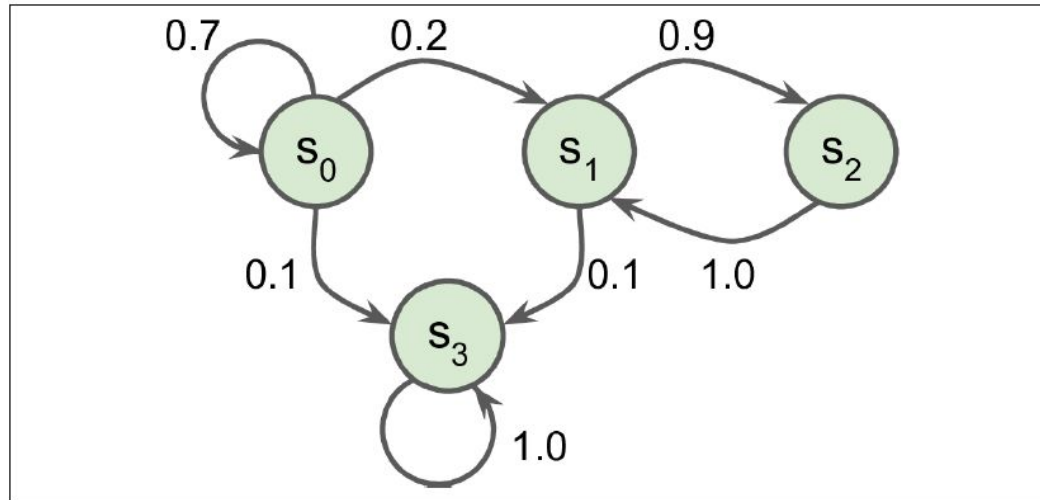


Figure 18-7. Example of a Markov chain

Markov Decision Processes (2)

- **Markov decision processes** (MDP) were first described in the 1950s.
- MDP resemble Markov chains but with a twist:
 - at each step, an agent can **choose one of several possible actions**, and
 - the **transition probabilities depend on the chosen action**.
 - **some state transitions return some reward** (positive or negative), and
 - the agent's goal is to find a policy that will **maximize reward over time**.

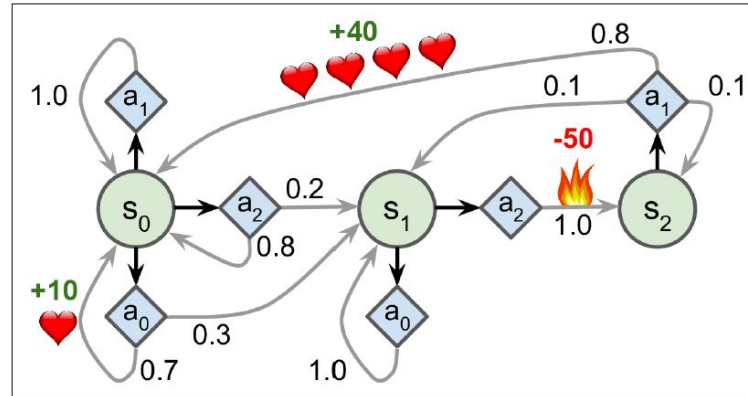


Figure 18-8. Example of a Markov decision process

Markov Decision Processes : Bellman Optimality Equation

- Estimate the optimal state value of any state s , noted $V^*(s)$, which is the sum of all discounted future rewards the agent can expect on average after it reaches a state s , assuming it acts optimally.
- If the agent acts optimally, then the *Bellman Optimality Equation* applies (see Equation 18-1).
- This recursive equation says that if the agent acts optimally, then the optimal value of the current state is equal to the reward it will get on average after taking one optimal action, plus the expected optimal value of all possible next states that this action can lead to.

Equation 18-1. Bellman Optimality Equation

$$V^*(s) = \max_a \sum_s T(s, a, s') [R(s, a, s') + \gamma \cdot V^*(s')] \quad \text{for all } s$$

Markov Decision Processes : Bellman Optimality Equation

Equation 18-1. Bellman Optimality Equation

$$V^*(s) = \max_a \sum_s T(s, a, s') [R(s, a, s') + \gamma \cdot V^*(s')] \quad \text{for all } s$$

- $T(s, a, s')$ is the transition probability from state s to state s' , given that the agent chose action a . For example, in Figure, $T(s_2, a_1, s_0) = 0.8$.
- $R(s, a, s')$ is the reward that the agent gets when it goes from state s to state s' , given that the agent chose action a . For example, in Figure, $R(s_2, a_1, s_0) = +40$.
- γ is the discount factor.

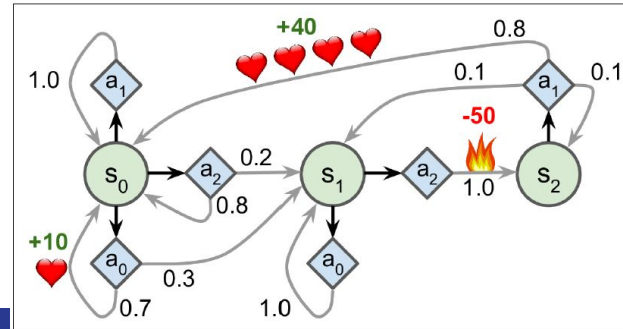


Figure 18-8. Example of a Markov decision process

MDP : Value Iteration algorithm

Equation 18-2. Value Iteration algorithm

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot V_k(s')] \quad \text{for all } s$$

where $V_k(s)$ is the estimated value of state s at the k^{th} iteration of the algorithm.

Algorithm that can precisely estimate the optimal state value of every possible state: you first initialize all the state value estimates to zero, and then you iteratively update them using the Value Iteration algorithm (see Equation).

Given enough time, these estimates are guaranteed to converge to the optimal state values, corresponding to the optimal policy.

MDP : Q-Value Iteration algorithm (1)

- Knowing the optimal state values can be useful, in particular to evaluate a policy, but it does not give us the optimal policy for the agent.
- An algorithm to estimate the optimal state-action values, generally called *Q-Values* (Quality Values).
- The optimal Q-Value of the state-action pair (s, a) , noted $Q^*(s, a)$, is the **sum of discounted future rewards the agent can expect on average after it reaches the state s and chooses action a , but before it sees the outcome of this action, assuming it acts optimally after that action.**

Equation 18-3. Q-Value Iteration algorithm

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \cdot \max_{a'} Q_k(s', a') \right] \quad \text{for all } (s, a)$$

MDP : Q-Value Iteration algorithm (2)

Once the optimal Q-Values are computed, defining the optimal policy, noted $\pi^*(s)$, is trivial: when the agent is in state s , it should choose the action with the highest Q-Value for that state:

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} Q^*(s, a).$$

MDP : Q-Value Iteration algorithm (3)

```

transition_probabilities = [ # shape=[s, a, s']
    [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
    [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
    [None, [0.8, 0.1, 0.1], None]]
rewards = [ # shape=[s, a, s']
    [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
    [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]]
possible_actions = [[0, 1, 2], [0, 2], [1]]

Q_values = np.full((3, 3), -np.inf) # -np.inf for impossible actions
for state, actions in enumerate(possible_actions):
    Q_values[state, actions] = 0.0 # for all possible actions

gamma = 0.90 # the discount factor

for iteration in range(50):
    Q_prev = Q_values.copy()
    for s in range(3):
        for a in possible_actions[s]:
            Q_values[s, a] = np.sum([
                transition_probabilities[s][a][sp]
                * (rewards[s][a][sp] + gamma * np.max(Q_prev[sp]))
            for sp in range(3)])

```

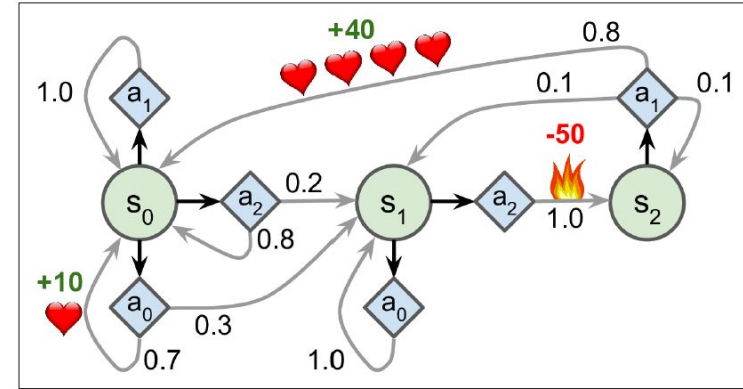


Figure 18-8. Example of a Markov decision process

```

>>> Q_values
array([[18.91891892, 17.02702702, 13.62162162],
       [ 0.          , -inf, -4.87971488],
       [ -inf, 50.13365013, -inf]])

```

```

>>> np.argmax(Q_values, axis=1) # optimal action for each state
array([0, 0, 1])

```

$s_0 : a_0, s_1 : a_0, s_2 : a_1$

Temporal Difference Learning (1)

Reinforcement Learning problems with discrete actions can often be modeled as Markov decision processes, but **the agent initially has no idea what the transition probabilities are** (it does not know $T(s, a, s')$), **and it does not know what the rewards are going to be either** (it does not know $R(s, a, s')$).

So, agent must experience each state and each transition at least once to know the rewards, and it must experience them multiple times if it is to have a reasonable estimate of the transition probabilities.

Temporal Difference Learning (2)

- The Temporal Difference Learning (TD Learning) takes into account the fact that the agent has only partial knowledge of the MDP.
- (In general we assume that the agent initially knows only the possible states and actions, and nothing more.)
- The agent uses an exploration policy—for example, a purely random policy—to explore the MDP, and as it progresses, the TD Learning algorithm updates the estimates of the state values based on the transitions and rewards that are actually observed:

Equation 18-4. TD Learning algorithm

$$V_{k+1}(s) \leftarrow (1 - \alpha)V_k(s) + \alpha(r + \gamma \cdot V_k(s'))$$

or, equivalently:

$$V_{k+1}(s) \leftarrow V_k(s) + \alpha \cdot \delta_k(s, r, s')$$

$$\text{with } \delta_k(s, r, s') = r + \gamma \cdot V_k(s') - V_k(s)$$

Temporal Difference Learning (3)

Equation 18-4. TD Learning algorithm

$$V_{k+1}(s) \leftarrow (1 - \alpha)V_k(s) + \alpha(r + \gamma \cdot V_k(s'))$$

or, equivalently:

$$V_{k+1}(s) \leftarrow V_k(s) + \alpha \cdot \delta_k(s, r, s')$$

$$\text{with } \delta_k(s, r, s') = r + \gamma \cdot V_k(s') - V_k(s)$$

where

- α is the learning rate (e.g., 0.01).
- $r + \gamma \cdot V_k(s')$ is called the *TD target*.
- $\delta_k(s, r, s')$ is called the *TD error*.

Q-Learning (1)

Q-Learning algorithm is an adaptation of the Q-Value Iteration algorithm to the situation where the transition probabilities and the rewards are initially unknown.

Equation 18-5. Q-Learning algorithm

$$Q(s, a) \leftarrow_{\alpha} r + \gamma \cdot \max_{a'} Q(s', a')$$

Q-Learning works by watching an agent play (e.g., randomly) and gradually improving its estimates of the Q-Values. Once it has accurate Q-Value estimates (or close enough), then the optimal policy is choosing the action that has the highest Q-Value (i.e., the greedy policy).

Q-Learning (2)

```
def step(state, action):
    probas = transition_probabilities[state][action]
    next_state = np.random.choice([0, 1, 2], p=probas)
    reward = rewards[state][action][next_state]
    return next_state, reward
```

```
def exploration_policy(state):
    return np.random.choice(possible_actions[state])
```

```
alpha0 = 0.05 # initial learning rate
decay = 0.005 # learning rate decay
gamma = 0.90 # discount factor
state = 0 # initial state
```

```
for iteration in range(10000):
    action = exploration_policy(state)
    next_state, reward = step(state, action)
    next_value = np.max(Q_values[next_state])
    alpha = alpha0 / (1 + iteration * decay)
    Q_values[state, action] *= 1 - alpha
    Q_values[state, action] += alpha * (reward + gamma * next_value)
    state = next_state
```

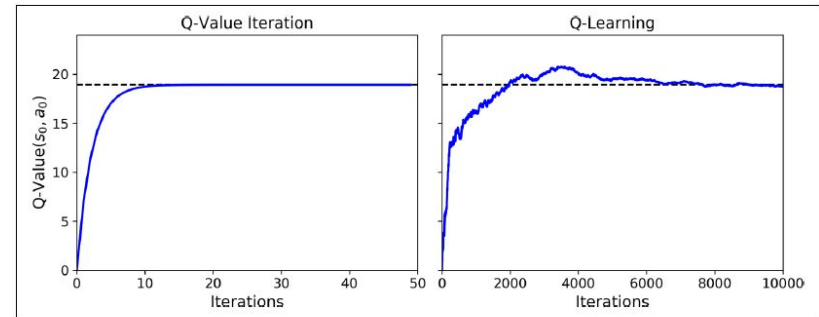


Figure 18-9. The Q-Value Iteration algorithm (left) versus the Q-Learning algorithm (right)

Exploration Policies (1)

- Q-Learning can work only if the exploration policy explores the MDP thoroughly enough.
- Although a purely random policy is guaranteed to eventually visit every state and every transition many times, it may take an extremely long time to do so.
- A better option is to use the ϵ -greedy policy (ϵ is epsilon): at each step it acts randomly with probability ϵ , or greedily with probability $1 - \epsilon$ (i.e., choosing the action with the highest Q-Value).
- Will spend **more and more time exploring the interesting parts of the environment**, as the Q-Value estimates get better and better.
- Common to start with a high value for ϵ (e.g., 1.0) and then gradually reduce it (e.g., down to 0.05).

Exploration Policies (2)

Rather than relying only on chance for exploration, another approach is to **encourage the exploration policy to try actions that it has not tried much before**. This can be implemented as a bonus added to the Q-Value estimates:

Equation 18-6. Q-Learning using an exploration function

$$Q(s, a) \leftarrow r + \gamma \cdot \max_{a'} f(Q(s', a'), N(s', a'))$$

where

- $N(s', a')$ counts the number of times the action a' was chosen in state s' .
- $f(Q, N)$ is an exploration function, such as $f(Q, N) = Q + \kappa/(1 + N)$, where κ is a curiosity hyperparameter that measures how much the agent is attracted to the unknown.

Approximate Q-Learning and Deep Q-Learning (1)

- Q-Learning does not scale well to large (or even medium) MDPs with many states and actions.
 - Ms. Pac-Man game: about 150 pellets that Ms. Pac-Man can eat, each of which can be present or absent (i.e., already eaten). So, the number of possible states is greater than $2^{150} \approx 10^{45}$. + positioning ghosts and MS. Pac-Man.
 - There's absolutely no way you can keep track of an estimate for every single Q-Value.
- *Approximate Q-Learning*: to find a function $Q_{\theta}(s, a)$ that approximates the Q-Value of any state-action pair (s, a) using a manageable number of parameters (given by the parameter vector θ).
 - DNN used to estimate Q-Values is called a **Deep Q-Network (DQN)**, and using a DQN for Approximate Q-Learning is called **Deep Q-Learning**.

Approximate Q-Learning and Deep Q-Learning (2)

How to train a DQN?

- Consider the approximate Q-Value computed by the DQN for a given state-action pair (s, a) . We want this approximate Q-Value to be **as close as possible to the reward r that we actually observe after playing action a in state s , plus the discounted value of playing optimally.**

Equation 18-7. Target Q-Value

$$Q_{\text{target}}(s, a) = r + \gamma \cdot \max_{a'} Q_{\theta}(s', a')$$

Implementing Deep Q-Learning

In theory, you need a neural net that takes a state-action pair and outputs an approximate Q-Value, but in practice it's much more efficient to use a **neural net that takes a state and outputs one approximate Q-Value for each possible action.**

```
env = gym.make("CartPole-v0")
input_shape = [4] # == env.observation_space.shape
n_outputs = 2 # == env.action_space.n

model = keras.models.Sequential([
    keras.layers.Dense(32, activation="elu", input_shape=input_shape),
    keras.layers.Dense(32, activation="elu"),
    keras.layers.Dense(n_outputs)
])
```

Implementing Deep Q-Learning : ϵ -greedy policy

Choose a random action with probability ϵ and store experiences in a replay buffer.

```
env = gym.make("CartPole-v0")
input_shape = [4] # == env.observation_space.shape
n_outputs = 2 # == env.action_space.n

model = keras.models.Sequential([
    keras.layers.Dense(32, activation="elu", input_shape=input_shape),
    keras.layers.Dense(32, activation="elu"),
    keras.layers.Dense(n_outputs)
])

from collections import deque

replay_buffer = deque(maxlen=2000)
```


Implementing Deep Q-Learning : Sample and step

```
def sample_experiences(batch_size):  
    indices = np.random.randint(len(replay_buffer), size=batch_size)  
    batch = [replay_buffer[index] for index in indices]  
    states, actions, rewards, next_states, done = [  
        np.array([experience[field_index] for experience in batch])  
        for field_index in range(5)]  
    return states, actions, rewards, next_states, done  
  
def play_one_step(env, state, epsilon):  
    action = epsilon_greedy_policy(state, epsilon)  
    next_state, reward, done, info = env.step(action)  
    replay_buffer.append((state, action, reward, next_state, done))  
    return next_state, reward, done, info
```

Implementing Deep Q-Learning : Training step

Sample a batch of experiences from the replay buffer and train the DQN

```
batch_size = 32
discount_factor = 0.95
optimizer = keras.optimizers.Adam(lr=1e-3)
loss_fn = keras.losses.mean_squared_error

def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, dones = experiences
    next_Q_values = model.predict(next_states)
    max_next_Q_values = np.max(next_Q_values, axis=1)
    target_Q_values = (rewards +
                       (1 - dones) * discount_factor * max_next_Q_values)
    mask = tf.one_hot(actions, n_outputs)
    with tf.GradientTape() as tape:
        all_Q_values = model(states)
        Q_values = tf.reduce_sum(all_Q_values * mask, axis=1, keepdims=True)
        loss = tf.reduce_mean(loss_fn(target_Q_values, Q_values))
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

Implementing Deep Q-Learning : Training

```
for episode in range(600):  
    obs = env.reset()  
    for step in range(200):  
        epsilon = max(1 - episode / 500, 0.01)  
        obs, reward, done, info = play_one_step(env, obs, epsilon)  
        if done:  
            break  
    if episode > 50:  
        training_step(batch_size)
```

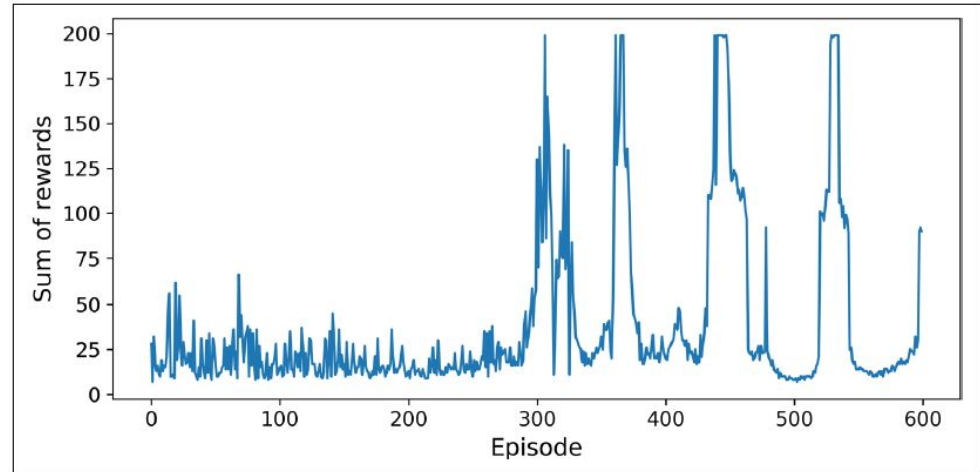


Figure 18-10. Learning curve of the Deep Q-Learning algorithm

Takeaways

- Estimating State-action values or Quality Values - Q-Values.
- Set of algorithms to estimate the MDP.
- Exploitation vs. Exploration, e.g. valuing unknown with a curiosity hyperparameter.
- Dealing with large state space for MDP with Deep Q-Learning.